

Deep Learning Exercise 1

Ron Hafzadi

Lior Kotlar

April 30th, 2025

Problem

You will find the training data at the course's moodle page. Split it to train and test sets randomly at a 1:9 ratio. We'd like to set up a small multi-layered perceptron (MLP) network to accept this data and output the proper prediction (detect / not detect).

Questions

a

How would you represent these 9-mers of amino acids (a vocabulary of 20 amino acids)? How would you represent the associate alleles? Explain your reasoning?

We decided to represent these 9-mers of amino acids as a matrix of size 20×9 , where each column corresponds to one of the 9 amino acid positions, and each column is a one-hot vector of length 20 representing one of the 20 amino acids. For implementation, we flatten this matrix into a 180-dimensional vector, allowing it to be fed directly into the MLP.

As for the alleles, we represent each allele as a separate one-hot vector. If there are 6 unique alleles, then each allele is represented as a vector of length 6 with a single 1 at the index corresponding to that allele.

If an antigen is positive for a given allele, we place a 1 in the corresponding index. If an antigen is not positive for any allele, the label vector contains all zeros.

b

What will the network's input dimension be under this representation choice? Implement an MLP that keeps this dimension for 2 inner layers, and plot the resulting train and test losses. Does this dimension pose a training problem?

The network's input dimension under this representation is 180, as each 9-mer antigen sequence is encoded as a flattened 20×9 one-hot matrix (i.e., 20 features for each of the 9 positions, resulting in 180 total input features).

We implemented a Multi-Layer Perceptron (MLP) with the following architecture:

- **Input layer:** 180-dimensional input vector
- **Hidden layer 1:** 180 units with ReLU activation
- **Hidden layer 2:** 180 units with ReLU activation
- **Output layer:** 6 units representing the alleles

We used the following hyper parameters:

- THRESHOLD = 0.3
- LR = 0.01
- N-EPOCHS = 30
- BATCH-SIZE = 16
- FALSE-NEG-WEIGHT = 3

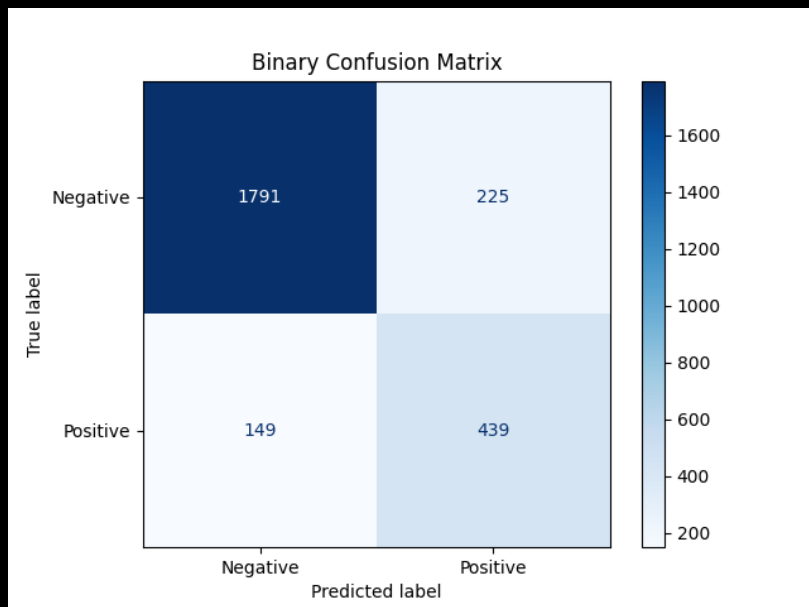
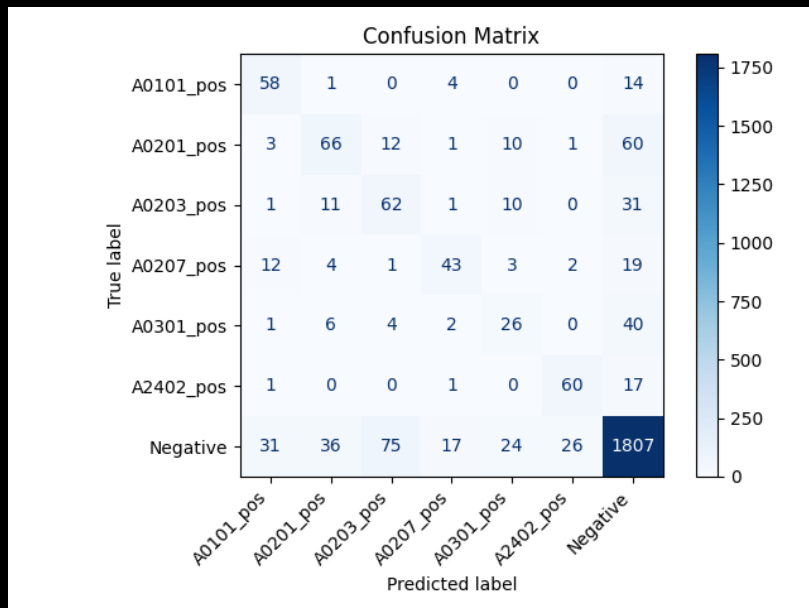
We trained the network and plotted both the training and test loss curves over epochs.

After training the network, we tested it by applying a threshold of 0.3 on the sigmoid outputs. We hypothesized that negative examples would produce relatively uniform activations across all alleles. Therefore, if none of the output values exceeded the threshold, we classified the antigen as negative. Conversely, if at least one allele's output surpassed the threshold, we considered the antigen as positively associated with that allele, therefore decided to return detected.

```
def decide(network_output, threshold):
    probabilities = torch.sigmoid(network_output)
    detection = torch.max(probabilities, dim=1)[0] > threshold
    return detection
```

Empirically, we found that the input dimensionality (180) is manageable. The model trains stably, and both train and test losses converge.

On the test set, we have reached an accuracy of 85.64, with precision of 0.6611 and recall of 0.7466.



c

Describe a network architecture that avoids the problems seen, explain all your design choices. Plot the train / tests losses. Include the plots obtained in your report.

We implemented a Multi-Layer Perceptron (MLP) with the following architecture:

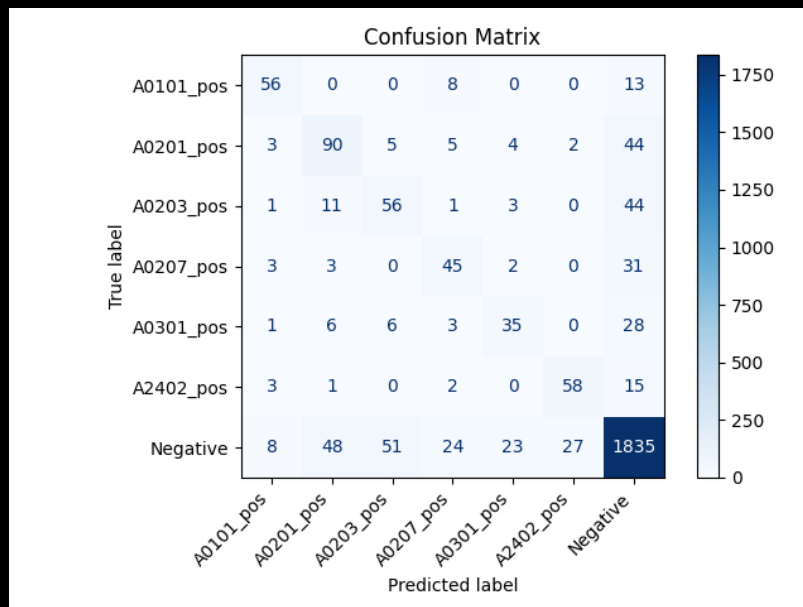
- **Input layer:** 180-dimensional input vector
- **Hidden layer 1:** 1024 units with ReLU activation
- **Hidden layer 2:** 1024 units with ReLU activation
- **Output layer:** 6 units representing the alleles

In order to methodically compare between the two networks, we pumped the hidden layers sizes to 1024. we kept all other Hyper-params the same.

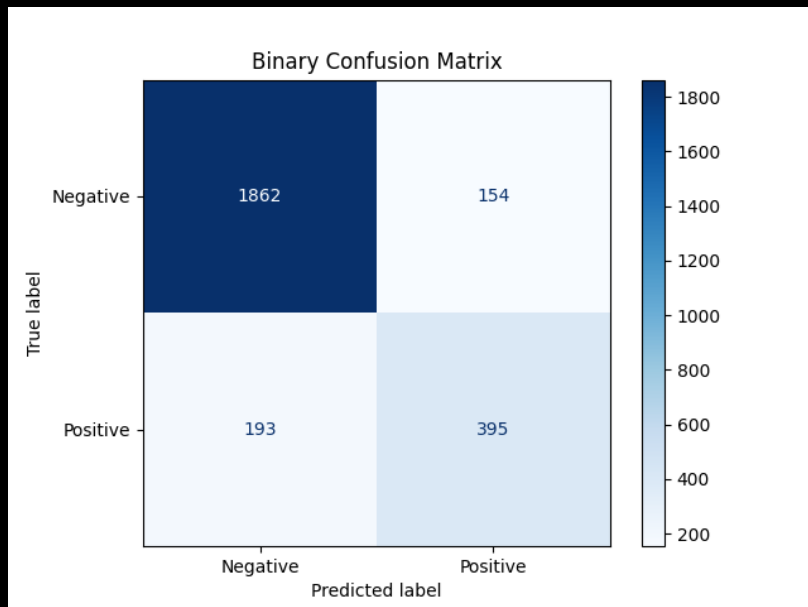
We trained the network and plotted both the training and test loss curves over epochs.



On the test set, we have reached an accuracy of 86.87, with precision of 0.7195 and recall of 0.6718.



When comparing the results of the model presented in section C and the model presented in section B, a worse test loss plot is seen in the C model graph. This can be attributed to overfitting. The overfitting arises from a more complex model that performs well on the training set but worse on the testing set.



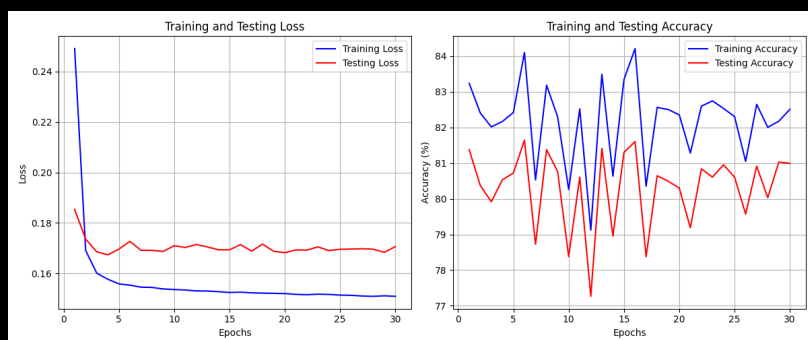
d

Remove the non-linear operators from your network, and report the results obtained. How do the results compare?

In this section, we took the same network of network c, and removed the relu activations.

We kept all other Hyper-params the same.

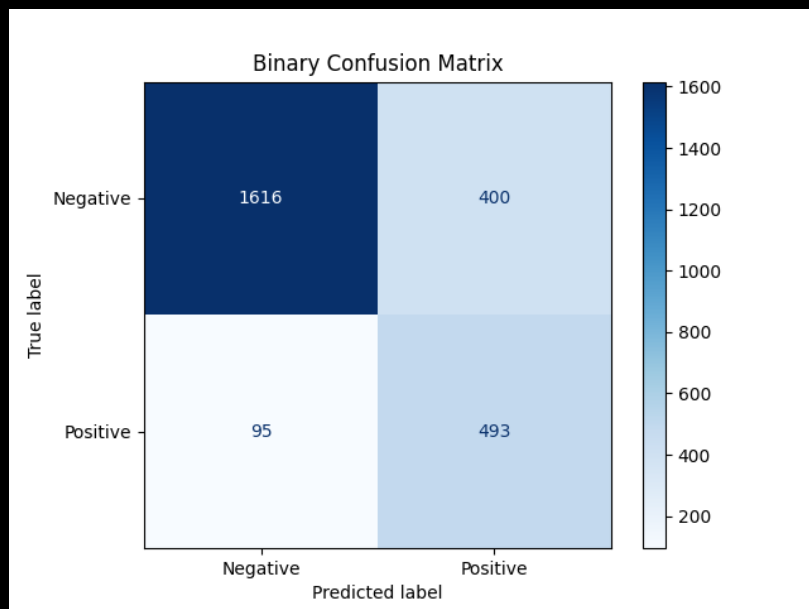
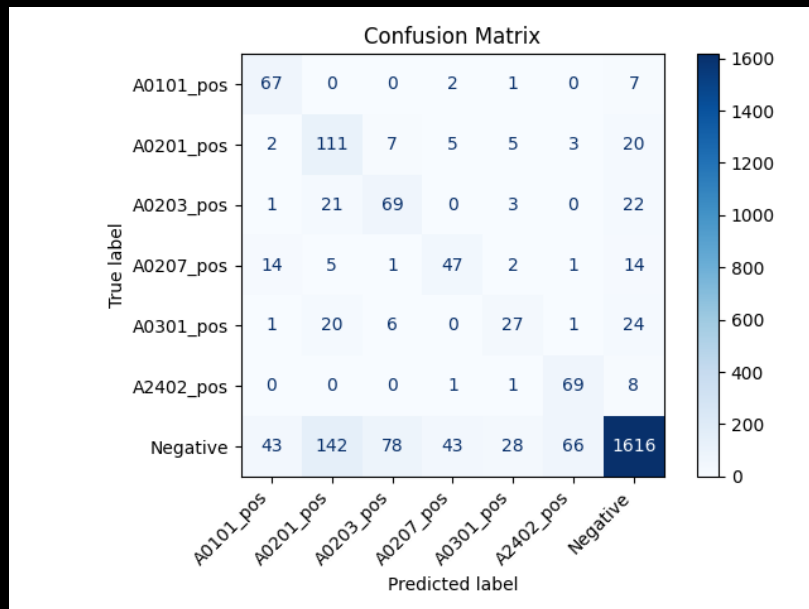
We trained the network and plotted both the training and test loss curves over epochs.



On the test set, we have reached an accuracy of 80.99, with precision of 0.5521 and recall of 0.8384.

Comparing the Linear Model to the Non-Linear Model

the linear model converged quickly but to a high loss, indicating underfitting due to limited capacity - it could not capture the complexity of the data. The non-linear model achieved a much lower training loss, showing better learning ability but its test loss increased after initial improvement indicating overfitting.



e

We used all 3 of our models to predict the detection of 9-amino-acids peptides from the Spike protein of the SARS-CoV-2.

MLP-B

Rank	Peptide	Allele	Score
1	MYICGDSTE	A2402_pos	12.9309
2	KYNENGTTT	A2402_pos	11.7229
3	NYKLPDDFT	A2402_pos	11.0774

MLP-C

Rank	Peptide	Allele	Score
1	KYNENGTTT	A2402_pos	11.5826
2	NYKLPDDFT	A2402_pos	10.6993
3	KWPWYIWLG	A2402_pos	8.6088

MLP-D

1	MYICGDSTE	A2402_pos	6.7252
2	YNENGTTTD	A0101_pos	6.5486
3	NYKLPDDFT	A2402_pos	6.1772

Preprocessing and Hyperparameters

Our first step was to remove duplicates, samples that were both in the negative and positive alleles, and samples that were in more than 1 positive allele were all removed to not harm the training process.

As the number of examples, and more importantly positive examples was sparse after cleaning, we decided to change our loss function to punish more for false negatives, which suits both the data distribution, and the fact that we prefer false positive than false negative when testing for illnesses. Hence, we decided that we punish times 3 when doing a false negative.

To identify the optimal hyperparameters, we performed an exhaustive search over a range of reasonable values for each model variant. Specifically, we varied the number of epochs, threshold, weights of positives, and batch size.

```
for epochs in (20, 30, 40, 50):
    for letter in ('B', 'C', 'D'):
        for threshold in (0.2, 0.3, 0.4, 0.5, 0.6):
            for weights in (1, 2, 3):
                for batchsize in (32, 16, 8):
                    if letter == 'B':
                        model = MLP_B()
                    elif letter == 'C':
                        model = MLP_C()
                    else:
                        model = MLP_D()
                    train_loader, test_loader = get_train_test(
                        alleles_dict, negative_antigens, weights, 0.1, batchsize)
                    model = train_model(model, train_loader, epochs)
                    test_model(model, test_loader, batchsize, threshold, weights, epochs)
```

Training with weights resulted in lower accuracy on the test set. After reflecting on the objectives of the task, we determined that accuracy alone is not the most appropriate metric. Therefore, we focused on optimizing recall, which better capture the model's ability to correctly identify true positives without being misled by the abundance of negative examples. Consequently, we selected the Hyper-params that maximized recall, rather than those that yielded the highest overall accuracy.